Class 7: Machine learning 1

Hyuntae An (TSN:200231511)

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionallity reduction**.

K-means

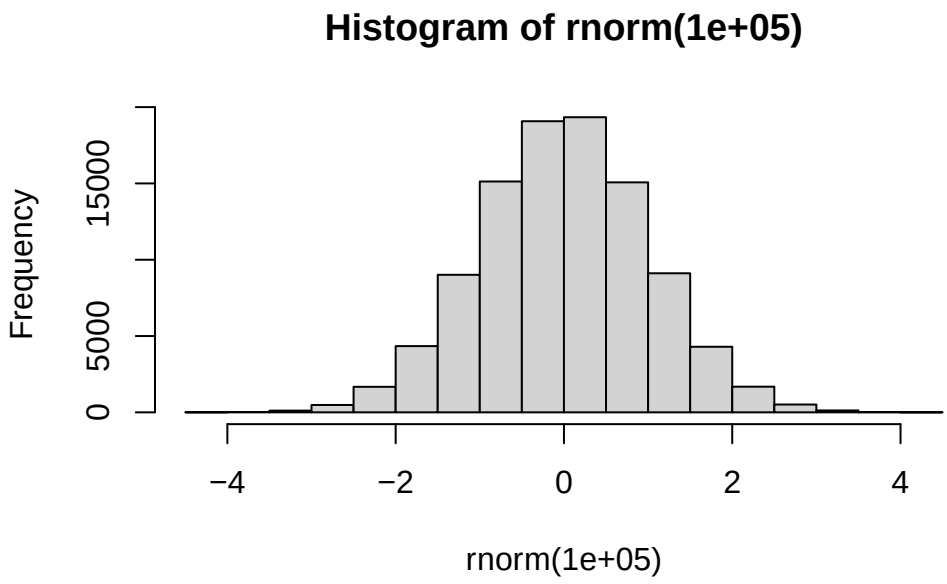
The main function in "base R" for K-means clustering is called `kmeans()`

Before we go too deep let's make up some "simple" data that we can cluster and know if we are getting good results or not. to do this we can use the `rnorm()` function:

```
rnorm(10)
```

```
 [1]  0.2488144 -1.2787395  1.3587773 -0.8225388  0.6360223 -0.4466526
 [7] -0.5125104 -0.1269010  1.9009990 -0.4301921
```

```
hist( rnorm(100000) )
```



Give me 30 points centered at -3 and a second set with a mean of +3

```
tmp <- c( rnorm(30, mean=-3), rnorm(30, mean=+3) )

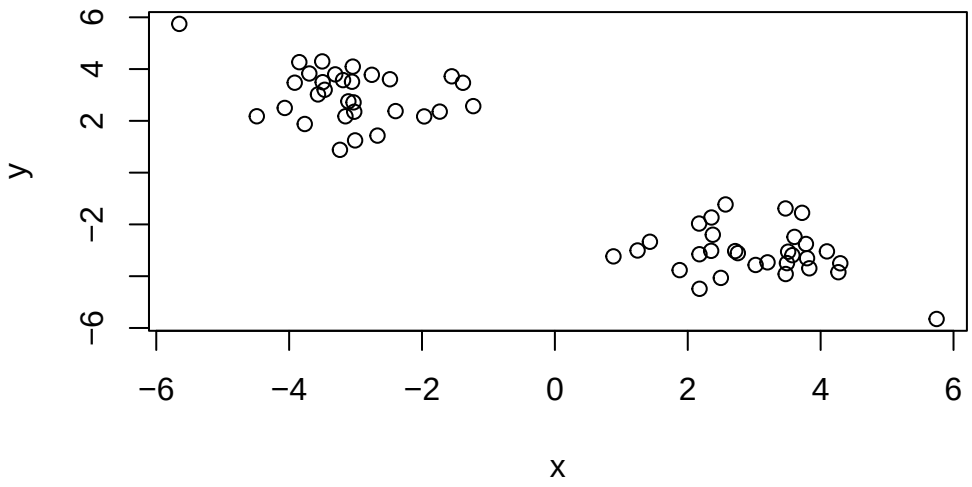
x <- cbind(x=tmp, y=rev(tmp))
x
```

```
               x          y
 [1,] -3.1885512  3.5740747
 [2,] -3.1524779  2.1761500
 [3,] -1.2288946  2.5686124
 [4,] -3.6958796  3.8294806
 [5,] -3.0535376  3.5108115
 [6,] -3.0072339  1.2445358
 [7,] -3.0201241  2.3519596
 [8,] -2.6699926  1.4303725
 [9,] -1.5522455  3.7205340
[10,] -4.4869183  2.1773131
[11,] -3.2350597  0.8814886
[12,] -4.0659553  2.4981584
[13,] -3.5016348  4.2959421
[14,] -2.3979090  2.3767462
[15,] -2.4823383  3.6066767
```

```
[16,] -3.4651110  3.2005490
[17,] -3.5661658  3.0212838
[18,] -3.0421069  4.0948586
[19,] -5.6535562  5.7456323
[20,] -2.7552239  3.7786237
[21,] -3.0304947  2.7137903
[22,] -3.3062244  3.7953645
[23,] -1.7331408  2.3566618
[24,] -3.4950692  3.4931257
[25,] -3.1086576  2.7531985
[26,] -1.9671754  2.1711205
[27,] -3.9160739  3.4736711
[28,] -1.3837872  3.4712836
[29,] -3.8468719  4.2667603
[30,] -3.7650047  1.8781137
[31,]  1.8781137 -3.7650047
[32,]  4.2667603 -3.8468719
[33,]  3.4712836 -1.3837872
[34,]  3.4736711 -3.9160739
[35,]  2.1711205 -1.9671754
[36,]  2.7531985 -3.1086576
[37,]  3.4931257 -3.4950692
[38,]  2.3566618 -1.7331408
[39,]  3.7953645 -3.3062244
[40,]  2.7137903 -3.0304947
[41,]  3.7786237 -2.7552239
[42,]  5.7456323 -5.6535562
[43,]  4.0948586 -3.0421069
[44,]  3.0212838 -3.5661658
[45,]  3.2005490 -3.4651110
[46,]  3.6066767 -2.4823383
[47,]  2.3767462 -2.3979090
[48,]  4.2959421 -3.5016348
[49,]  2.4981584 -4.0659553
[50,]  0.8814886 -3.2350597
[51,]  2.1773131 -4.4869183
[52,]  3.7205340 -1.5522455
[53,]  1.4303725 -2.6699926
[54,]  2.3519596 -3.0201241
[55,]  1.2445358 -3.0072339
[56,]  3.5108115 -3.0535376
[57,]  3.8294806 -3.6958796
[58,]  2.5686124 -1.2288946
```

```
[59,]  2.1761500 -3.1524779
[60,]  3.5740747 -3.1885512
```

```
plot(x)
```



Now try `kmeans()` on this input data `x`

```
k <- kmeans(x, centers = 2)
k
```

```
K-means clustering with 2 clusters of sizes 30, 30

Cluster means:
          x         y
1  3.015230 -3.092447
2 -3.092447  3.015230

Clustering vector:
 [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Within cluster sum of squares by cluster:
```

```
[1] 56.62027 56.62027
 (between_SS / total_SS =  90.8 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

To get to the results we can use `k$size` etc. to access each component result:

Q. How many points are in each cluster?

```
k$size
```

```
[1] 30 30
```

Q. What "component" of your results object details cluster memebership?

```
k$cluster
```

```
 [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

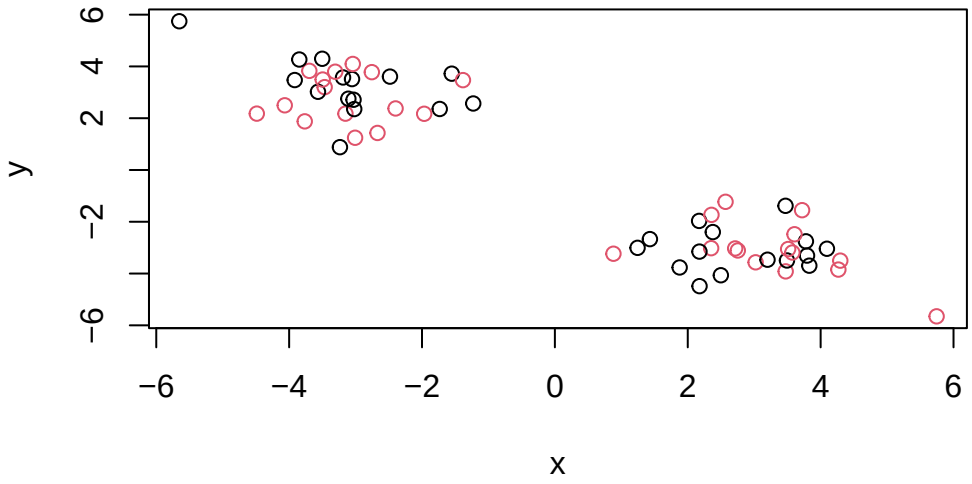
Q. What "component" of your results object details cluster centers?

```
k$centers
```

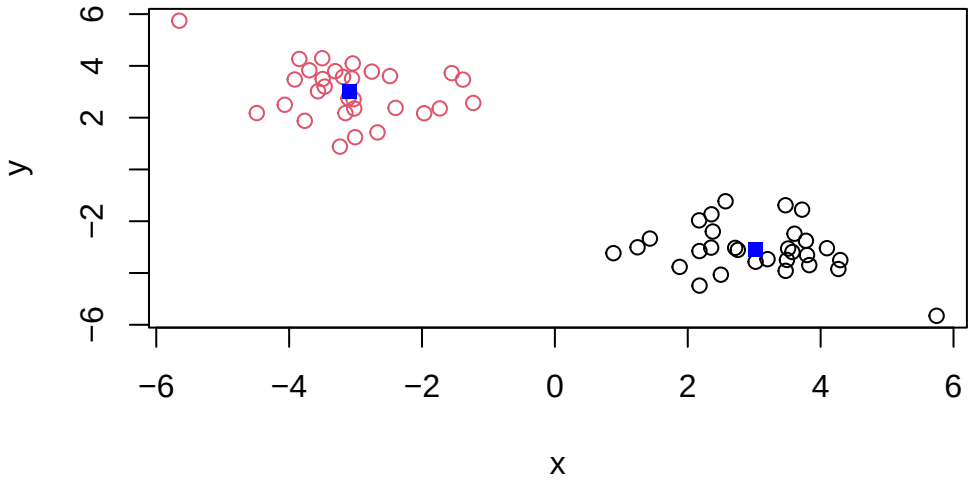
```
          x         y
1  3.015230 -3.092447
2 -3.092447  3.015230
```

Q. Make a results figure of our data with poitnts colored by cluster along with cluster centers.

```
plot(x, col= c(1,2))
```

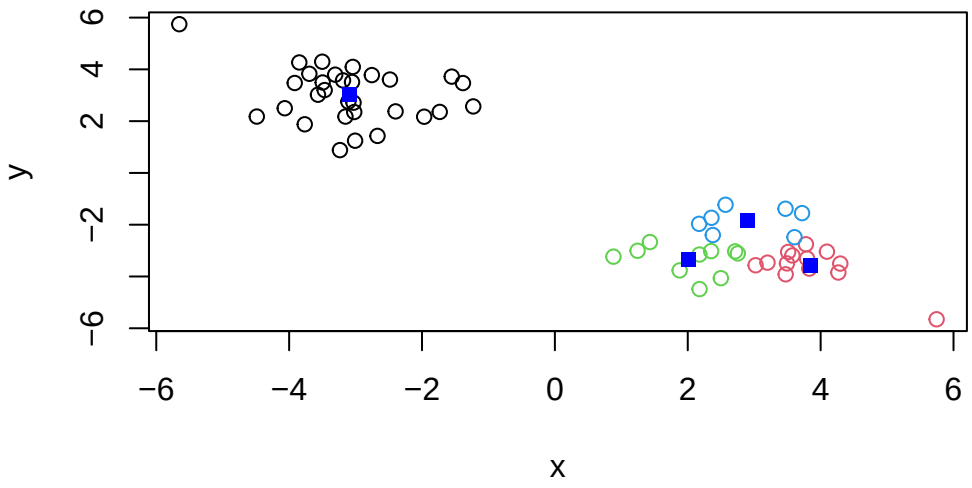



```
plot(x,col=k$cluster)
points(k$centers, col="blue",pch=15)
```



Q. Run kmeans again and cluster into 4 clusters producing a result figure with points colored by cluster and showing the cluster centers

```
km <- kmeans(x, centers = 4)
plot(x,col=km$cluster)
points(km$centers, col="blue",pch=15)
```



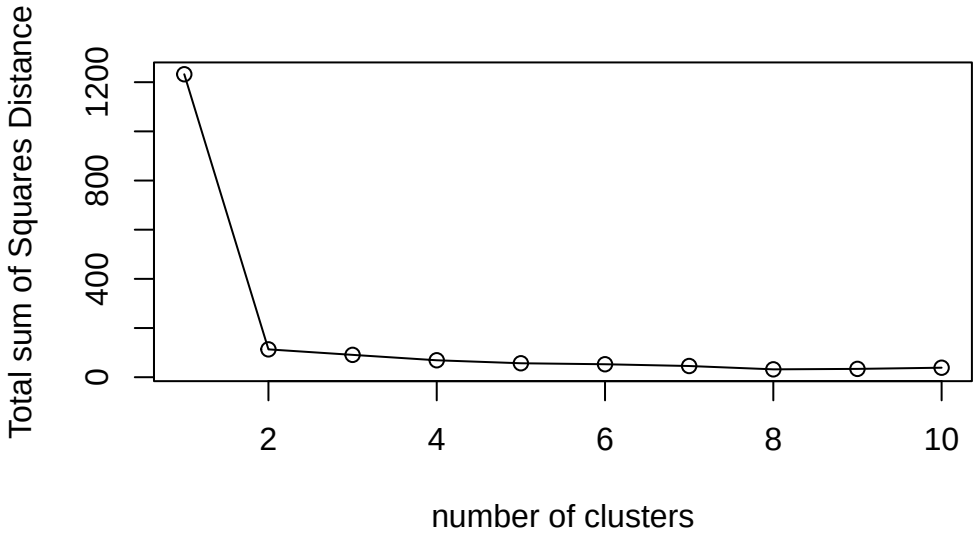
N.B. You need to tell k-means the number of clusters (i.e. set `centers=2`)!!

One approach is to try different values for `centers`and then pick the best one:

```
ans <- NULL

for(i in 1:10) {
  #cat(i,"\n")
  km <- kmeans(x,centers = i)
  ans <- c(ans, km$tot.withinss)
}
```

```
plot(ans, typ="o",
     xlab= "number of clusters",
     ylab= "Total sum of Squares Distance")
```



Hierarchical Clustering

The main function in "base" R for Hierarchical Clustering is called `hclust()`

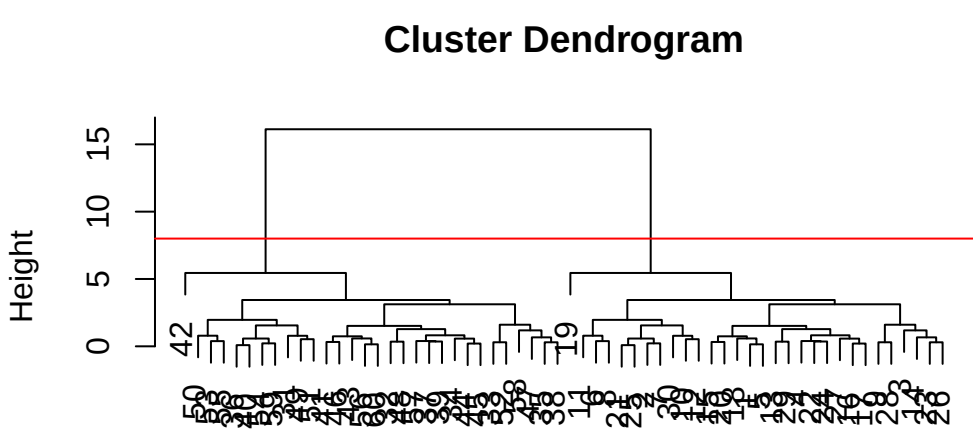
This function does not take your "raw" data for clustering. You must first build a "distance matrix" from your data and pass this as input for `hclust()`.

```
d <- dist(x)
hc <- hclust(d)
hc
```

```
Call:
hclust(d = d)

Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

```
plot(hc)
abline (h=8, col="red")
```



Once we have our `hclust` object (our "tree" or "cluster dendrogram") we can "cut" the tree to reveral the clustering pattern:

```
cutree(hc, h=8)
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

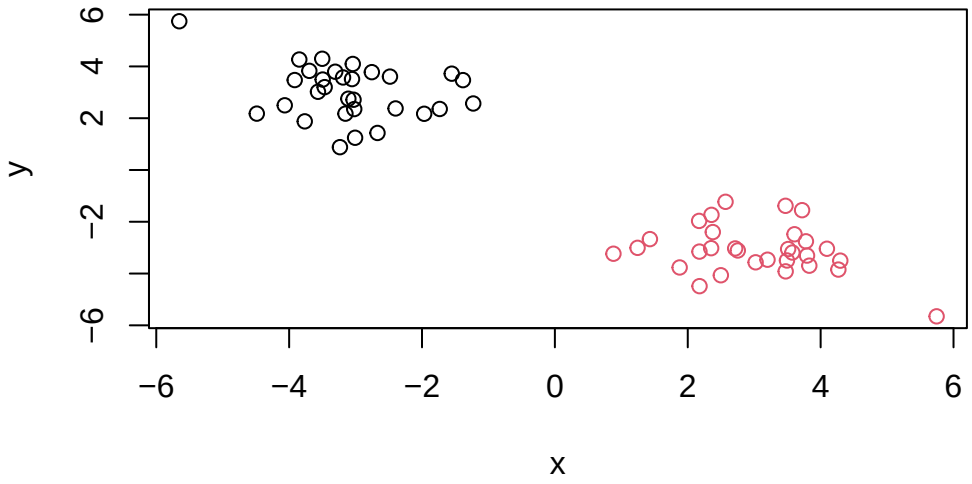
You can use either `h=` or `k=` arguments to `cutree()`

```
cutree(hc, k=4)
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3
[39] 3 3 3 4 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
```

> Q. Make a plot of input data `x` with your clusters from `hclust()` (i.e. colored by cluster membership from cutting the tree)?

```
grps <- cutree(hc,h=8)
plot(x, col = grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

let's look at some data on eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data import

The data is made available in CSV format so we can use `read.csv()`:

```
x <- read.csv("https://tinyurl.com/UK-foods", row.names = 1)
x
```

```
                England Wales Scotland N.Ireland
Cheese              105   103      103        66
Carcass_meat        245   227      242       267
```

```
Other_meat             685   803      750       586
Fish                   147   160      122        93
Fats_and_oils          193   235      184       209
Sugars                 156   175      147       139
Fresh_potatoes         720   874      566      1033
Fresh_Veg              253   265      171       143
Other_Veg              488   570      418       355
Processed_potatoes     198   203      220       187
Processed_Veg          360   365      337       334
Fresh_fruit           1102  1137      957       674
Cereals               1472  1582     1462      1494
Beverages               57    73       53        47
Soft_drinks           1374  1256     1572      1506
Alcoholic_drinks       375   475      458       135
Confectionery           54    64       62        41
```

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
nrow(x)
```

```
[1] 17
```

```
ncol(x)
```

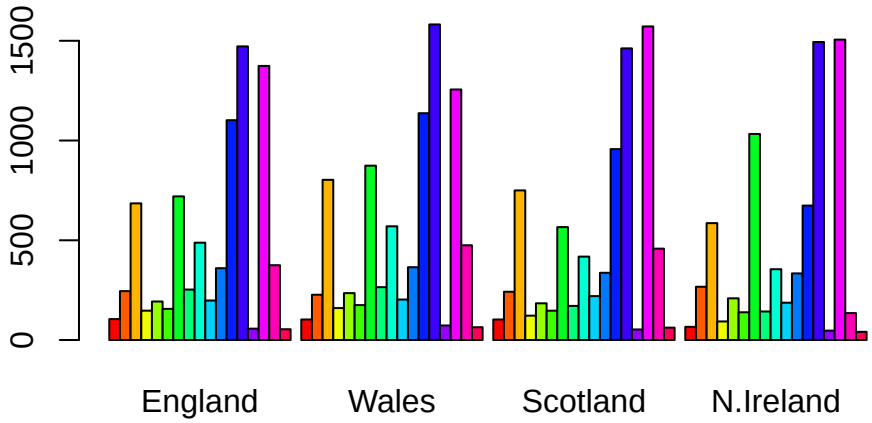
```
[1] 4
```

Q2. Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

Setting row.names=1 inside read.csv() is the better choice. The other approach sets rownames(x) <- x[,1] and then runs x <- x[,-1], and that is destructive.

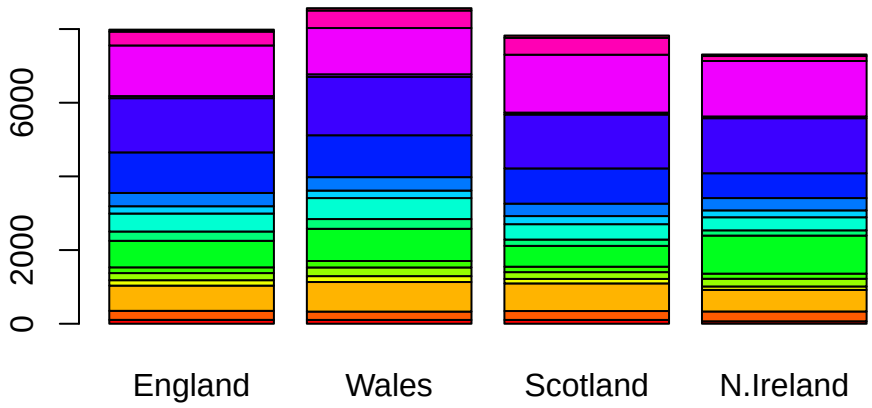
Some plots:

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3.Changing what optional argument in the above barplot() function results in the following plot?

```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



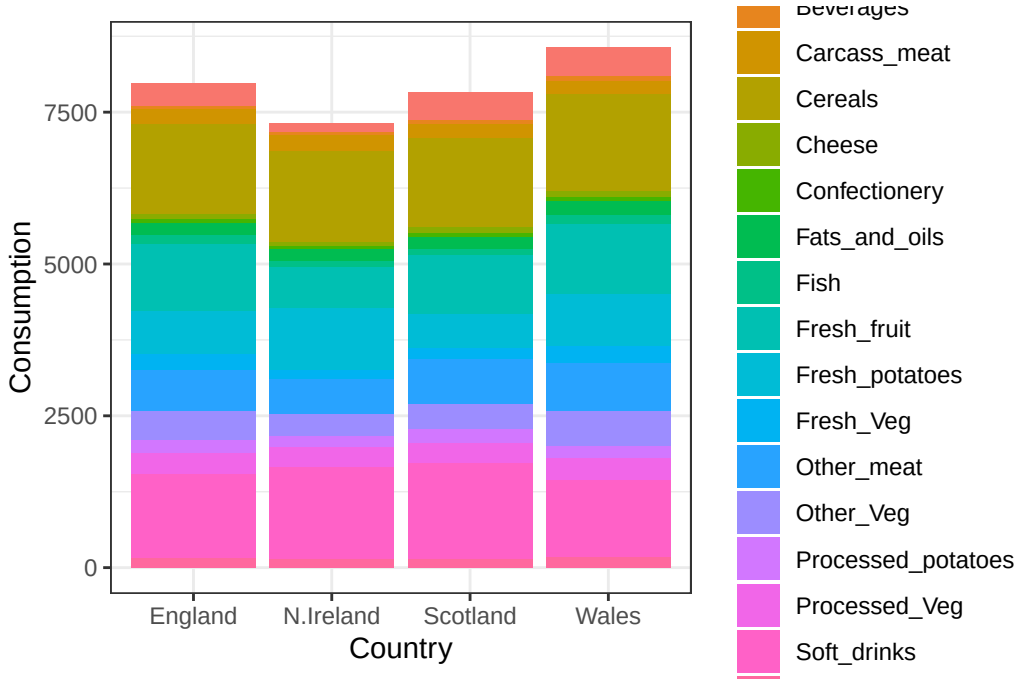
Changing beside = TRUE to beside = FALSE (or F) turns the grouped barplot into the stacked one you have here.

> Q4. Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

```
library(tidyr)
library(ggplot2)

x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food, names_to = "Country", values_to = "Consumption")

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



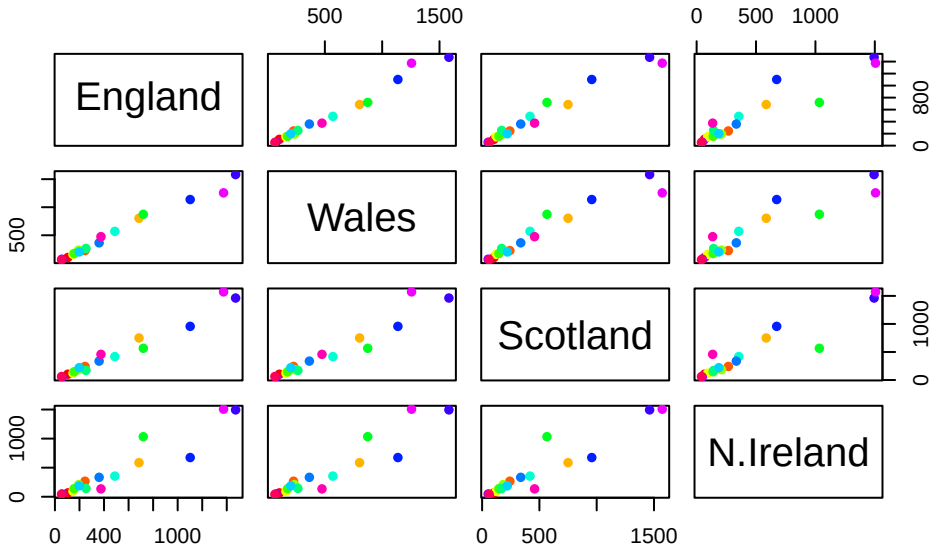
Change position = "dodge" to position = "stack"

Pairs plots and heatmaps

One plot that can be useful for relatively small datasets is a `pairs()` plot:

Q5. We can use the pairs() function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



The two countries eat about the same amount of that food. A point above the diagonal means the y-axis country eats more of it, and a point below means the x-axis country eats more. England, Wales and Scotland plotted against each other sit fairly close to the diagonal, so their diets are broadly similar, with only a little scatter.

Key-point: Even relatively small datasets can prove challenging to interpret

Given that it is quite difficult to make sense of even this relatively small data set. Hopefully, we can clearly see that a powerful analytical method is absolutely necessary if we wish to observe trends and patterns in larger datasets.

PCA to the rescue!

The main function in "base" R for PCA is called `prcomp()`:

This function will require the transpose of our dataset

```
pca <- prcomp( t(x) )
```

We can see how well or PCA results are doing in terms of the "Variance captured" from the original data in each PC.

```
summary(pca)
```

```
Importance of components:
                            PC1      PC2      PC3       PC4
Standard deviation     324.1502 212.7478 73.87622 2.921e-14
Proportion of Variance   0.6744   0.2905  0.03503 0.000e+00
Cumulative Proportion    0.6744   0.9650  1.00000 1.000e+00
```

I can see what is in my `pca` results object by:

```
attributes(pca)
```

```
$names
[1] "sdev"     "rotation" "center"   "scale"    "x"

$class
[1] "prcomp"
```

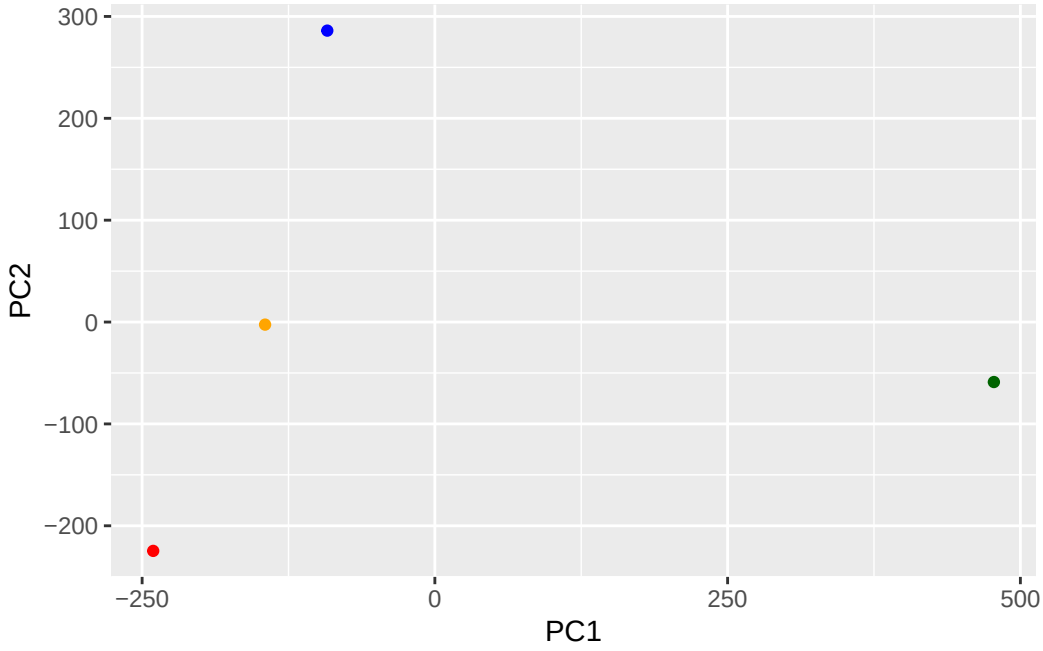
Of these `pca$x` is our main result. We can use it to make a results figure often called a PCA plot (or score- plot).

```
pca$x
```

```
                PC1         PC2        PC3           PC4
England   -144.99315   -2.532999 105.768945 -9.152022e-15
Wales     -240.52915 -224.646925 -56.475555  5.560040e-13
Scotland   -91.86934  286.081786 -44.415495 -6.638419e-13
N.Ireland  477.39164  -58.901862  -4.877895  1.329771e-13
```

```
library(ggplot2)

ggplot(pca$x) +
  aes(PC1,PC2) +
  geom_point(col=c("orange","red","blue","darkgreen"))
```

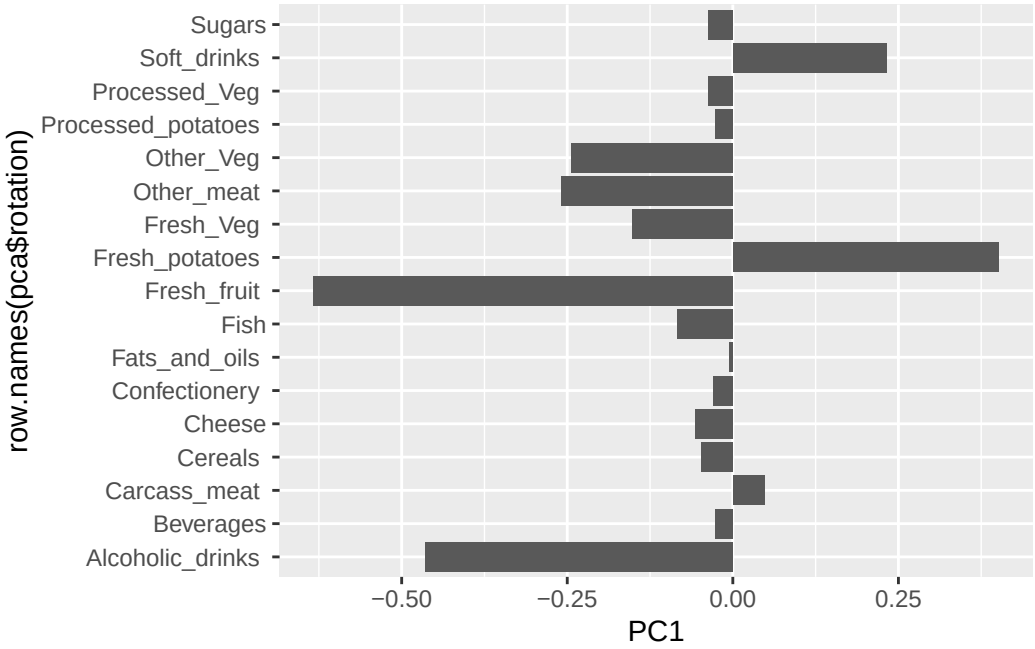


A "loadings" plot can show us how the original dimensions/variables contribute to our new PC axis. This is detailed in the `pca$rotation` component of our new result object:

```
pca$rotation
```

	PC1	PC2	PC3	PC4
Cheese	-0.056955380	0.016012850	0.02394295	-0.409382587
Carcass_meat	0.047927628	0.013915823	0.06367111	0.729481922
Other_meat	-0.258916658	-0.015331138	-0.55384854	0.331001134
Fish	-0.084414983	-0.050754947	0.03906481	0.022375878
Fats_and_oils	-0.005193623	-0.095388656	-0.12522257	0.034512161
Sugars	-0.037620983	-0.043021699	-0.03605745	0.024943337
Fresh_potatoes	0.401402060	-0.715017078	-0.20668248	0.021396007
Fresh_Veg	-0.151849942	-0.144900268	0.21382237	0.001606882
Other_Veg	-0.243593729	-0.225450923	-0.05332841	0.031153231
Processed_potatoes	-0.026886233	0.042850761	-0.07364902	-0.017379680
Processed_Veg	-0.036488269	-0.045451802	0.05289191	0.021250980
Fresh_fruit	-0.632640898	-0.177740743	0.40012865	0.227657348
Cereals	-0.047702858	-0.212599678	-0.35884921	0.100043319
Beverages	-0.026187756	-0.030560542	-0.04135860	-0.018382072
Soft_drinks	0.232244140	0.555124311	-0.16942648	0.222319484
Alcoholic_drinks	-0.463968168	0.113536523	-0.49858320	-0.273126013
Confectionery	-0.029650201	0.005949921	-0.05232164	0.001890737

```
ggplot(pca$rotation) +
  aes(PC1,row.names(pca$rotation)) +
  geom_col()
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

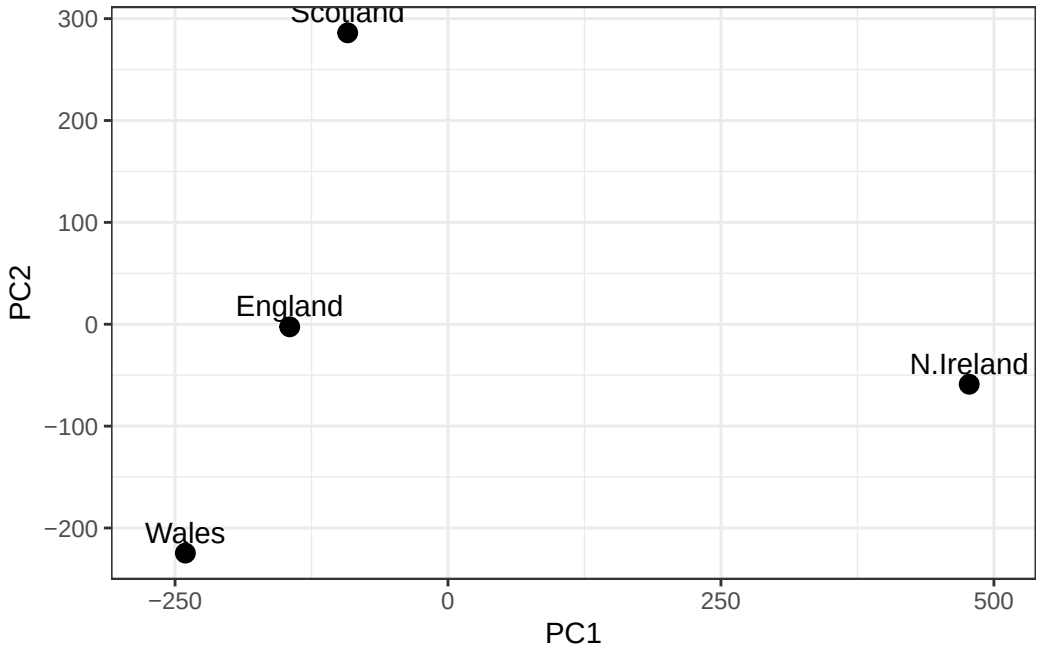
England, Wales and Scotland cluster together, and N. Ireland is the outlier in both the pairs plot and the heatmap dendrogram. This suggests the three have similar diets while N. Ireland's differs.

It is hard to say exactly how they differ from these plots alone. You can spot hints, such as more fresh potatoes and less fresh fruit and alcohol in NI, but it isn't easy to pick out the main drivers. That difficulty is the reason the lab moves on to PCA.

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
df <- as.data.frame(pca$x)
df$Country <- rownames(df)
```

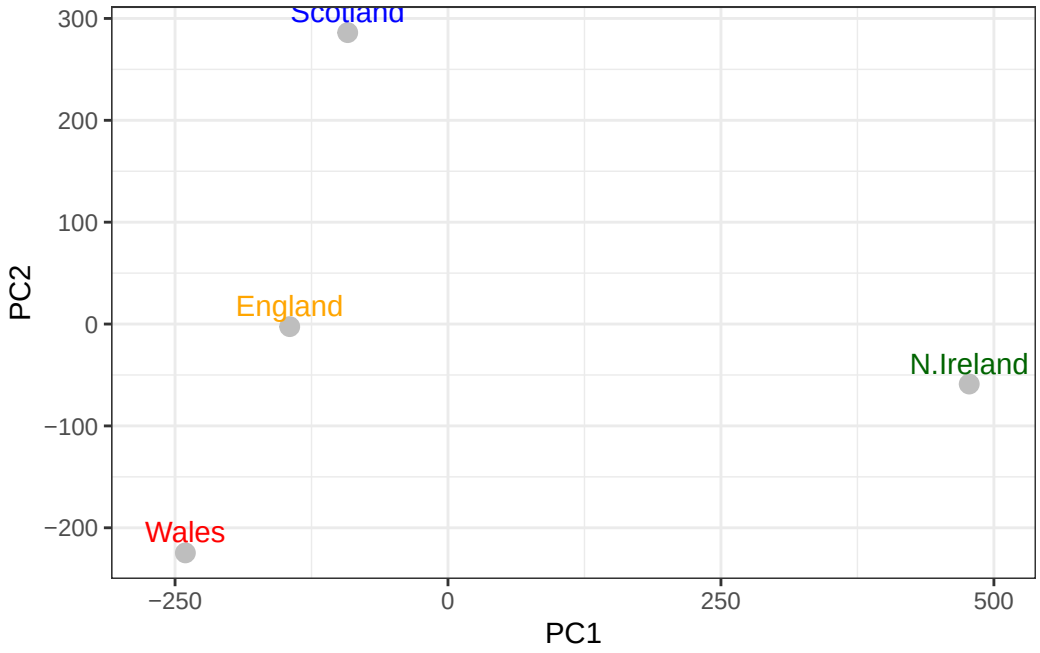
```
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) + xlab("PC1") + ylab("PC2") + theme_bw()
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
mycols <- c("orange", "red", "blue", "darkgreen")  # England, Wales, Scotland, N.Ireland

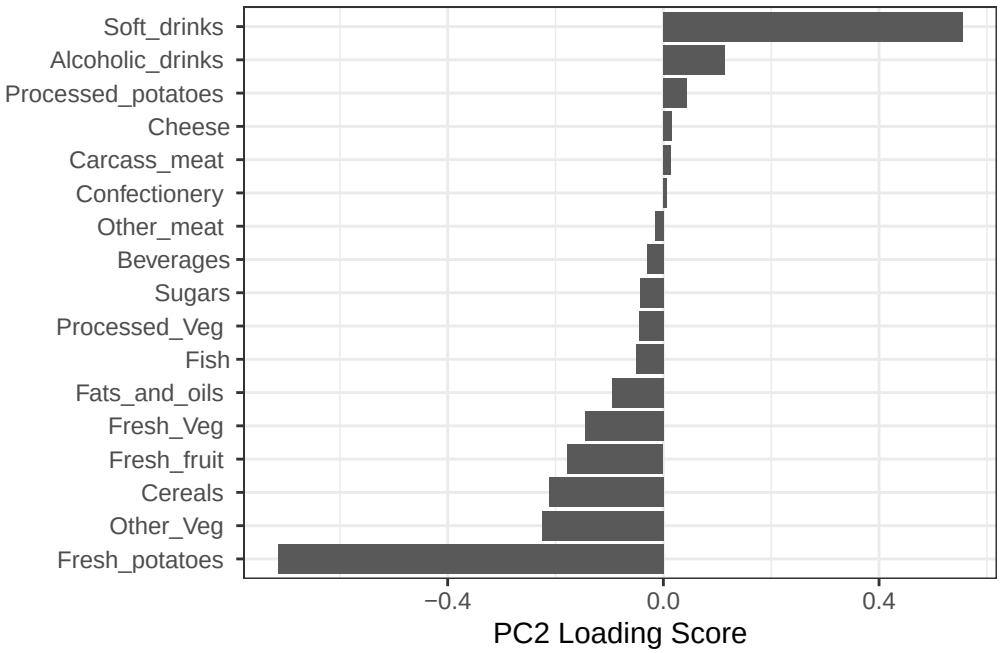
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country) +
  geom_point(size = 3, col = "grey") +
  geom_text(vjust = -0.5, col = mycols) +
  xlim(-270, 500) + xlab("PC1") + ylab("PC2") + theme_bw()
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominantely and what does PC2 maninly tell us about?

```
load2 <- as.data.frame(pca$rotation)
load2$Food <- rownames(load2)

ggplot(load2) +
  aes(PC2, reorder(Food, PC2)) +
  geom_col() + xlab("PC2 Loading Score") + ylab("") + theme_bw()
```



Soft drinks has by far the largest positive PC2 loading. The bar cut off at the bottom is Fresh potatoes, with the largest negative loading. Scroll down or enlarge the plot window to see its label.